**Indian Institute of Technology Hyderabad**

Department of Computer Science and Engineering

*CS6160: Advanced Topics in Cryptology*

---

# Programming Assignment 1 — Problem 4

Zero-Knowledge Proof of Knowledge and
Correct Evaluation of a Secret Polynomial

---

**Group Member 1:** Divyansh Sevta
**Roll Number:** CS25MTECH14018
**Group Member 2:** Nishanth D.
**Roll Number:** CS25MTECH14020

**Instructor:** Dr. Maria Francis **Semester:** Spring 2026

# Contents

**1 Introduction** **2**

**2 Preliminaries** **2**
2.1 Cyclic group and discrete log assumption . . . 2
2.2 Pedersen commitment . . . 2

**3 Problem Statement** **3**

**4 Why Naive Aggregation Fails** **3**

**5 The Parallel Per-Coefficient Σ-Protocol** **3**
5.1 Protocol . . . 3
5.2 Why the evaluation check binds . . . 4

**6 Fiat–Shamir Non-Interactive Version** **4**

**7 Security Analysis** **5**

**8 Implementation** **5**
8.1 File layout . . . 6
8.2 Core Σ-protocol . . . 6
8.3 Fiat–Shamir challenge . . . 6
8.4 Two interactive variants . . . 7

**9 Input Generator** **7**
9.1 Miller–Rabin primality test . . . 7
9.2 Safe-prime search and generators . . . 8
9.3 Parameter-size flag . . . 8

**10 Testing and Results** **8**
10.1 End-to-end test matrix . . . 8
10.2 Concrete toy example . . . 9
10.3 REJECT example . . . 9
10.4 Reproduction . . . 10

**11 Security Summary** **10**

**12 Conclusion** **10**

**Abstract**

We design and implement a zero-knowledge proof system by which a prover convinces a verifier that (i) the prover knows the coefficients $a_0, \ldots, a_d \in \mathbb{Z}_q$ of a secret polynomial $P(x)$ that match public Pedersen commitments $C_0, \ldots, C_d$, and (ii) $P(z) \equiv y \pmod q$ for a publicly supplied point $z$ and claimed value $y$. The protocol is a parallel per-coefficient $\Sigma$-protocol with a single shared challenge, augmented by a scalar evaluation-consistency check that binds the opening of each commitment to the claimed evaluation. We prove completeness, 2-special-soundness with a full-witness extractor, and perfect honest-verifier zero-knowledge. Both a multi-round interactive protocol and a Fiat–Shamir non-interactive variant are implemented in pure Python 3 with only `hashlib`, `secrets`, and standard built-ins; no external cryptographic library is used.

# 1 Introduction

Polynomials are a fundamental object in modern cryptography: polynomial commitments (KZG, Bulletproofs, FRI) underpin SNARKs, STARKs, and verifiable secret-sharing. In this assignment we consider the simplest instance of this pattern. The prover holds a secret polynomial of degree $d$ over $\mathbb{Z}_q$; the public parameters include Pedersen commitments to each coefficient and a challenge point $z$. The prover must convince a verifier that the polynomial evaluates to a claimed value $y$ at $z$, without revealing any coefficient.

Our construction is deliberately elementary: we do not rely on pairings or on polynomial interpolation tricks. Instead we run $d+1$ parallel Schnorr proofs of knowledge of the individual Pedersen openings, share a single challenge across them, and attach one extra scalar equation that binds the coefficients to $y$. This yields an efficient $\Sigma$-protocol that provably meets the assignment's soundness requirement: from two accepting transcripts with distinct challenges, one can extract the *full* witness $(a_0, \ldots, a_d, r_0, \ldots, r_d)$.

# 2 Preliminaries

## 2.1 Cyclic group and discrete log assumption

Let $p$ be a prime and $q$ a prime divisor of $p-1$. Let $G_q \subseteq \mathbb{Z}_p^*$ be the unique subgroup of order $q$ with public generator $g$. We additionally fix an *independent* generator $h \in G_q$ such that $\log_g h$ is unknown. All exponent arithmetic is in $\mathbb{Z}_q$; all group-element arithmetic is in $\mathbb{Z}_p^*$.

**Definition 1** (Discrete Logarithm Problem)**.** *Given $g, y \in G_q$, find $x \in \mathbb{Z}_q$ with $g^x = y \pmod p$.*

We assume DLP is hard in $G_q$. The binding property of the Pedersen commitment below reduces to this assumption.

## 2.2 Pedersen commitment

**Definition 2** (Pedersen Commitment)**.** *For $v \in \mathbb{Z}_q$ and $r \xleftarrow{\$} \mathbb{Z}_q$,*

$$\mathrm{Com}(v;\, r) \;=\; g^v\, h^r \pmod p.$$

**Perfect hiding.** The map $r \mapsto h^r$ is a bijection from $\mathbb{Z}_q$ onto $G_q$. For any fixed $v$, the distribution of $g^v h^r$ over uniform $r$ is the uniform distribution on $G_q$, independent of $v$. Hence Pedersen is *perfectly* hiding.

**Computational binding.** Suppose an adversary opens a single commitment $C$ two ways: $C = g^a h^r = g^{a'} h^{r'}$ with $(a, r) \neq (a', r')$. Then $g^{a-a'} = h^{r'-r}$, giving $\log_g h = (a - a')(r' - r)^{-1} \bmod q$. Under the DL assumption this is infeasible, so Pedersen commitments are computationally binding.

# 3 Problem Statement

**Public inputs.** $(G_q, q, g, h)$; polynomial degree $d$; coefficient commitments $C_0, \ldots, C_d$; evaluation point $z \in \mathbb{Z}_q$; claimed value $y \in \mathbb{Z}_q$; rounds $k$.

**Witness.** $\mathbf{w} = (a_0, \ldots, a_d,\ r_0, \ldots, r_d) \in \mathbb{Z}_q^{2(d+1)}$.

**Relation.**

$$\mathcal{R} \;=\; \Big\{\big((C_0, \ldots, C_d, z, y);\ \mathbf{w}\big) \;:\; C_i = g^{a_i} h^{r_i}\ \forall i,\ \textstyle\sum_{i=0}^{d} a_i z^i \equiv y \pmod q\Big\}.$$

A valid protocol must satisfy completeness, computational soundness, and (honest-verifier) zero-knowledge. A knowledge-sound protocol, moreover, must admit a *knowledge extractor* that, given sufficient transcripts, outputs a witness in $\mathcal{R}$.

# 4 Why Naive Aggregation Fails

A tempting shortcut is to aggregate commitments as $C_{\text{eval}} = \prod_i C_i^{z^i}$, observe that $C_{\text{eval}} = g^{P(z)} h^R$ with $R = \sum_i r_i z^i$, and run a single Schnorr PoK of $R$ over $D = C_{\text{eval}} \cdot g^{-y}$.

The extractor for this single-aggregate protocol outputs only the scalar $\hat{R} = \sum_i r_i z^i$; it cannot recover any individual $a_i$ or $r_i$. The assignment's statement "I know coefficients corresponding to $C_0, \ldots, C_d$" therefore fails the proof-of-knowledge requirement. We fix this by running *parallel* openings in a single combined $\Sigma$-protocol, while still enforcing the evaluation constraint as a scalar equation on the responses.

# 5 The Parallel Per-Coefficient $\Sigma$-Protocol

## 5.1 Protocol

**Statements verified:** (1) $g^{a_i} h^{r_i} = C_i$ for all $i$; (2) $\sum_{i=0}^{d} a_i z^i \equiv y \pmod q$.

| **Prover** | | **Verifier** |
|---|---|---|
| For $i = 0, \ldots, d$: $k_{i,a}, k_{i,r} \xleftarrow{\$} \mathbb{Z}_q$ | | |
| $T_i \leftarrow g^{k_{i,a}} h^{k_{i,r}} \pmod p$ | | |
| $E \leftarrow \sum_{i=0}^{d} k_{i,a}\, z^i \pmod q$ | | |
| send $(T_0, \ldots, T_d,\, E)$ | $\longrightarrow$ | |
| | $\xleftarrow{c}$ | $c \xleftarrow{\$} \mathbb{Z}_q$ |
| For each $i$: | | |
| $u_{i,a} = k_{i,a} + c \cdot a_i \pmod q$ | | |
| $u_{i,r} = k_{i,r} + c \cdot r_i \pmod q$ | | |
| send $\{(u_{i,a}, u_{i,r})\}_{i=0}^{d}$ | $\longrightarrow$ | |
| | | **Check (V1):** for all $i$, |
| | | $g^{u_{i,a}} h^{u_{i,r}} \stackrel{?}{=} T_i \cdot C_i^c \pmod p$ |
| | | **Check (V_eval):** |
| | | $\sum_i u_{i,a} z^i \stackrel{?}{\equiv} E + c \cdot y \pmod q$ |
| | | Accept iff both checks pass. |

Repeat independently $k$ times with fresh nonces to amplify soundness.

### 5.2 Why the evaluation check binds

Computing $\sum_i u_{i,a}\, z^i$ honestly gives

$$\sum_{i=0}^{d} u_{i,a}\, z^i = \sum_{i=0}^{d} (k_{i,a} + c\, a_i)\, z^i = \underbrace{\sum_i k_{i,a}\, z^i}_{=E} + c \underbrace{\sum_i a_i\, z^i}_{=P(z)}.$$

Check (V_eval) therefore passes iff $P(z) = y$. Crucially, $E$ is committed in the first message, before $c$ is chosen, so the prover cannot adapt it.

**Does $E$ leak?** $E = \sum_i k_{i,a} z^i$ is a fixed linear combination of independent uniform random variables in $\mathbb{Z}_q$; it is uniformly distributed on $\mathbb{Z}_q$, independent of the coefficients $a_i$.

## 6 Fiat–Shamir Non-Interactive Version

Replace the verifier's random challenge with a hash of every public element and the first message:

$$c = H\big(\mathsf{tag} \,\|\, g \,\|\, h \,\|\, p \,\|\, q \,\|\, C_0 \,\|\, \cdots \,\|\, C_d \,\|\, z \,\|\, y \,\|\, T_0 \,\|\, \cdots \,\|\, T_d \,\|\, E\big) \bmod q,$$

where $H =$ SHA-256, $\mathsf{tag}$ is a protocol identifier, and each field is serialised as a 4-byte big-endian length prefix followed by its big-endian byte representation. Length prefixing removes encoding ambiguity; the tag provides domain separation.

**Security in the ROM.** Security follows from the Forking Lemma of Pointcheval–Stern [**?**]: a successful forger can be rewound to produce two accepting transcripts sharing the same

$(T_0, \ldots, T_d, E)$ but distinct challenges, from which the extractor of Section 7 recovers the full witness, contradicting either binding or DL hardness.

# 7 Security Analysis

**Theorem 1** (Completeness)**.** *An honest prover holding $(a_i, r_i)$ with $P(z) = y$ is always accepted.*

*Proof.* **(V1):** $g^{u_{i,a}} h^{u_{i,r}} = g^{k_{i,a}+ca_i} h^{k_{i,r}+cr_i} = (g^{k_{i,a}} h^{k_{i,r}})(g^{a_i} h^{r_i})^c = T_i \cdot C_i^c$. **(V_eval):** $\sum_i u_{i,a} z^i = \sum_i k_{i,a} z^i + c \sum_i a_i z^i = E + c \cdot P(z) = E + cy$. □

**Theorem 2** (Special Soundness — Full Witness Extraction)**.** *Given two accepting transcripts that share the same first message $(T_0, \ldots, T_d, E)$ but have distinct challenges $c \neq c'$:*

$$(T_*, E, c, \{(u_{i,a}, u_{i,r})\}) \quad \textit{and} \quad (T_*, E, c', \{(u'_{i,a}, u'_{i,r})\}),$$

*one can efficiently extract the full witness $(\hat{a}_0, \ldots, \hat{a}_d, \hat{r}_0, \ldots, \hat{r}_d)$ with $C_i = g^{\hat{a}_i} h^{\hat{r}_i}$ for all $i$ and $\sum_i \hat{a}_i z^i = y$.*

*Proof.* Let $\delta = (c - c') \in \mathbb{Z}_q^*$. From (V1) on both transcripts: $g^{u_{i,a}-u'_{i,a}} h^{u_{i,r}-u'_{i,r}} = C_i^{c-c'} = C_i^{\delta}$. Setting

$$\hat{a}_i = (u_{i,a} - u'_{i,a})\, \delta^{-1}, \qquad \hat{r}_i = (u_{i,r} - u'_{i,r})\, \delta^{-1} \pmod{q}$$

gives $g^{\hat{a}_i} h^{\hat{r}_i} = C_i$. Subtracting the two (V_eval) equations yields $\sum_i (u_{i,a} - u'_{i,a}) z^i = (c - c')y = \delta y$; dividing by $\delta$ gives $\sum_i \hat{a}_i z^i = y$. The extractor outputs all $2(d+1)$ witness components. □

The soundness error per round is $1/q$, negligible in the bit-length of $q$; $k$ independent rounds reduce it to $1/q^k$.

**Theorem 3** (Perfect Honest-Verifier Zero-Knowledge)**.** *A PPT simulator without the witness produces transcripts identically distributed to real transcripts.*

*Proof sketch.* Simulator: pick $c \xleftarrow{\$} \mathbb{Z}_q$; for each $i$ pick $u_{i,a}, u_{i,r} \xleftarrow{\$} \mathbb{Z}_q$; define $T_i \leftarrow g^{u_{i,a}} h^{u_{i,r}} C_i^{-c}$; define $E \leftarrow \sum_i u_{i,a} z^i - cy$. Output $(T_i, E, c, \{(u_{i,a}, u_{i,r})\})$. Both (V1) and (V_eval) hold by construction. In a real run the responses $u_{i,a} = k_{i,a} + ca_i$ are uniform and independent of $c$ because $k_{i,a}$ is; $T_i$ and $E$ are uniquely determined by $(c, u_{i,a}, u_{i,r}, C_i, y)$ via the same formulae as the simulator. Hence real and simulated distributions are identical. □

# 8 Implementation

All code is Python 3. The only standard-library imports are `hashlib`, `secrets`, `random`, `json`, `os`, `sys`, and `time`. No external cryptographic or ZK library is used, per the assignment constraint.

### 8.1 File layout

| File | Role |
|---|---|
| `generate_inputs.py` | Input generator (primes, generators, witness) |
| `Problem4.py` | NI and interactive-hash prover/verifier |
| `prover_interactive.py` | Live two-program prover (Variant B) |
| `verifier_interactive.py` | Live two-program verifier (Variant B) |
| `public.json` | Public inputs (read by both sides) |
| `private.json` | Prover's witness |
| `proof.json` | Proof transcript (prover → verifier) |

### 8.2 Core Σ-protocol

Listing 1: Commit, response, and verification.

```
def commit_phase(d, z, g, h, p, q):
    k_a = [secrets.randbelow(q) for _ in range(d + 1)]
    k_r = [secrets.randbelow(q) for _ in range(d + 1)]
    T_list = [(pow(g, ka, p) * pow(h, kr, p)) % p
              for ka, kr in zip(k_a, k_r)]
    E, zp = 0, 1
    for ka in k_a:
        E = (E + ka * zp) % q
        zp = (zp * z) % q
    return T_list, E, k_a, k_r

def response_phase(c, coeffs, randomness, k_a, k_r, q):
    u_a = [(ka + c * a) % q for ka, a in zip(k_a, coeffs)]
    u_r = [(kr + c * r) % q for kr, r in zip(k_r, randomness)]
    return u_a, u_r

def verify_checks(C_list, z, y, T_list, E, c, u_a, u_r, g, h, p, q):
    for C, T, ua, ur in zip(C_list, T_list, u_a, u_r):
        lhs = (pow(g, ua, p) * pow(h, ur, p)) % p
        rhs = (T * pow(C, c, p)) % p
        if lhs != rhs:
            return False
    lhs_eval, zp = 0, 1
    for ua in u_a:
        lhs_eval = (lhs_eval + ua * zp) % q
        zp = (zp * z) % q
    return lhs_eval == (E + c * y) % q
```

### 8.3 Fiat–Shamir challenge

Listing 2: Canonical length-prefixed SHA-256 challenge.

```
def _enc_int(n):
    raw = n.to_bytes((n.bit_length() + 7) // 8 or 1, "big")
```

```
    return len(raw).to_bytes(4, "big") + raw

def fs_challenge_ni(g, h, p, q, commitments, z, y, T_list, E):
    H = hashlib.sha256()
    H.update(b"ZKP-POLY-FS-v1:")
    for v in (g, h, p, q):
        H.update(_enc_int(v))
    H.update(_enc_int(len(commitments)))
    for C in commitments:
        H.update(_enc_int(C))
    H.update(_enc_int(z))
    H.update(_enc_int(y))
    for T in T_list:
        H.update(_enc_int(T))
    H.update(_enc_int(E))
    return int.from_bytes(H.digest(), "big") % q
```

### 8.4 Two interactive variants

**Variant A — hash-derived per-round challenge.** A single program runs $k$ rounds, using a separate domain tag and round index in the hash:

$$c_j = H(\text{"ZKP-POLY-INT-v1 : "} \,\|\, j \,\|\, \text{public} \,\|\, T_0^{(j)} \,\|\, \ldots \,\|\, T_d^{(j)} \,\|\, E^{(j)}) \bmod q.$$

The verifier recomputes $c_j$ independently, so a malicious prover cannot grind it. This matches the soundness of the Fiat–Shamir variant per round.

**Variant B — live two-program exchange.** The prover and verifier are separate processes communicating via a file-based message queue under `./session/`. Per round $j$ the prover writes `round_j_A.json` (commit), the verifier samples $c_j \xleftarrow{\$} \mathbb{Z}_q$ and writes `round_j_B.json` (challenge), the prover reads it and writes `round_j_C.json` (response), and the verifier writes `round_j_V.json` (per-round verdict). This faithfully realises the classical $\Sigma$-protocol message flow.

## 9 Input Generator

`generate_inputs.py` produces `public.json` and `private.json` entirely from scratch. Allowed imports are `hashlib` (SHA-256 only), `secrets`, `random`, `json`, and `sys`; all cryptographic primitives are hand-written.

### 9.1 Miller–Rabin primality test

```
def is_prime(n):
    if n < 2: return False
    for p in _SMALL_PRIMES:
        if n == p: return True
        if n % p == 0: return False
    d, r = n - 1, 0
```

```
    while d % 2 == 0:
        d //= 2; r += 1
    bases = _DETERMINISTIC_BASES if n < (1 << 82) \
            else [2 + secrets.randbelow(n - 3) for _ in range(40)]
    for a in bases:
        if a % n == 0: continue
        x = pow(a, d, n)
        if x == 1 or x == n - 1: continue
        composite = True
        for _ in range(r - 1):
            x = x * x % n
            if x == n - 1: composite = False; break
        if composite: return False
    return True
```

Using the deterministic base set $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$ the test is *exact* for all $n < 3.3 \cdot 10^{24}$; above that we fall back to 40 random bases (error probability $\leq 2^{-80}$).

### 9.2 Safe-prime search and generators

We sample a random $b$-bit odd integer $q$, prefilter by small-prime trial division on both $q$ and $p = 2q + 1$, then Miller–Rabin-test each. A generator of the order-$q$ subgroup is obtained as $g = x^2 \bmod p$ for random $x$, verifying $g \neq 1$ and $g^q \equiv 1$.

The second generator $h$ is produced by one of two user-selected methods:

`-h-mode hash` Hash-to-subgroup: $h = x^2 \bmod p$ where $x = \mathrm{SHA256}(\text{"ZKP-POLY-H-v1"} \| \mathrm{ctr}) \bmod p$, with counter incremented until the result has order $q$ and is distinct from $\{0, 1, g\}$. No one (prover, verifier, or designer) knows $\log_g h$.

`-h-mode discard` Sample $s \xleftarrow{\$} \mathbb{Z}_q^*$, set $h = g^s \bmod p$, then discard $s$. This is trapdoor in principle (whoever ran the generator once knew $s$), so the hash-to-subgroup method is preferred in production.

### 9.3 Parameter-size flag

| Flag | Size of $q$ | Purpose |
|---|---|---|
| `-size toy` | $q = 101$ (PDF's example) | Human-readable grading |
| `-size small` | 128 bits | Fast end-to-end runs |
| `-size full` | 256 bits | Production demo |

## 10 Testing and Results

### 10.1 End-to-end test matrix

All six {toy, small, full} × {hash, discard} combinations produce valid inputs and pass every proof mode. A tampered proof (where the first response scalar is shifted by $+1 \bmod q$) is correctly rejected in every case.

| size | h-mode | NI | interactive-hash | NI-tampered |
|---|---|---|---|---|
| toy | hash | ACCEPT | ACCEPT | REJECT |
| toy | discard | ACCEPT | ACCEPT | REJECT |
| small | hash | ACCEPT | ACCEPT | REJECT |
| small | discard | ACCEPT | ACCEPT | REJECT |
| full | hash | ACCEPT | ACCEPT | REJECT |
| full | discard | ACCEPT | ACCEPT | REJECT |

The live two-program variant (`prover_interactive.py` / `verifier_interactive.py`) was also exercised with $k = 4$ rounds on 128-bit parameters and produced ACCEPT.

### 10.2 Concrete toy example

Listing 3: `public.json` (toy)

```
{
  "p": 607, "q": 101, "g": 7, "h": 49,
  "d": 2, "k": 3,
  "commitments": [346, 270, 552],
  "z": 9, "y": 5
}
```

Listing 4: `private.json` (toy)

```
{ "coeffs": [33, 92, 63],
  "randomness": [70, 39, 10] }
```

Verification: $P(9) = 33 + 92 \cdot 9 + 63 \cdot 81 = 33 + 828 + 5103 = 5964 \equiv 5 \pmod{101}$. ✓

Listing 5: `proof.json` (non-interactive, after running `Problem4.py prover -mode noninteractive`)

```
{
  "mode": "noninteractive",
  "transcript": {
    "T_list": [552, 212, 26],
    "E": 9,
    "c": 55,
    "u_a": [18, 54, 11],
    "u_r": [43, 36, 22]
  },
  "tampered": false
}
```

**Verifier output:** `ACCEPT`. Manually check (V_eval): $\sum_i u_{i,a} \cdot 9^i = 18 + 54 \cdot 9 + 11 \cdot 81 = 18 + 486 + 891 = 1395 \equiv 82 \pmod{101}$, and $E + cy = 9 + 55 \cdot 5 = 284 \equiv 82 \pmod{101}$. ✓

### 10.3 REJECT example

Re-running the prover with `-tamper` shifts $u_{0,a}$ by $+1 \bmod q$. The verifier prints `REJECT`.

### 10.4 Reproduction

```
# 1. generate inputs
python generate_inputs.py --size small --h-mode hash \
                        --degree 3 --rounds 4

# 2. non-interactive proof
python Problem4.py prover --mode noninteractive
python Problem4.py verifier # ACCEPT

# 3. single-program interactive (hash-derived challenge)
python Problem4.py prover --mode interactive-hash
python Problem4.py verifier # ACCEPT

# 4. live two-program interactive
python verifier_interactive.py & # shell A
python prover_interactive.py # shell B

# 5. tamper demo
python Problem4.py prover --mode noninteractive --tamper
python Problem4.py verifier # REJECT
```

## 11 Security Summary

| Property | Statement |
|---|---|
| Completeness | Honest $(a_i, r_i)$ with $P(z) = y$ always pass (V1) and (V_eval). |
| Special Soundness | Two transcripts with $(c \neq c')$ on same first message extract the *full* $2(d+1)$-dim witness. |
| Perfect HVZK | Real and simulated transcripts are identically distributed. |
| Evaluation Binding | (V_eval) forces $\sum_i a_i z^i = y$; cheating requires either a false $E$ (caught by extraction) or breaking DL. |
| Non-interactive | Fiat–Shamir in ROM yields a NIZK of equivalent security via the Forking Lemma. |

## 12 Conclusion

We delivered a from-scratch implementation of a zero-knowledge proof of knowledge and correct evaluation of a secret polynomial, satisfying all requirements of the assignment: Pedersen commitments, a multi-round interactive $\Sigma$-protocol, a Fiat–Shamir non-interactive variant, rigorous completeness / special-soundness / HVZK proofs, and an input generator built without external cryptographic libraries. The design extracts the *full* witness (not just an aggregated linear combination), which strictly meets the proof-of-knowledge standard.

## References

1. T. P. Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing," *CRYPTO 1991*.
2. C. P. Schnorr, "Efficient Identification and Signatures for Smart Cards," *CRYPTO 1989*.
3. M. Bellare and P. Rogaway, "Random Oracles are Practical," *ACM CCS 1993*.
4. D. Pointcheval and J. Stern, "Security Proofs for Signature Schemes," *Journal of Cryptology*, 2000.
5. D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, 2023.

## Anti-Plagiarism Statement

We certify that this submission is entirely our own work. All references are cited appropriately. No external cryptographic library was used.

**Names:** Divyansh Sevta, Nishanth D. **Date:** April 20, 2026